

PARA O ALTO E AVANTE

PERÍCIA FORENSE POST-MORTEM



5

LISTA DE FIGURAS

Figura 1 – Representação dos arquivos em disco	8
Figura 2 – Imagem de referência	9
Figura 3 – Análise da imagem de referência.....	9
Figura 4 – Assinatura do arquivo imagem de referência	10
Figura 5 – Verificação do <i>magic number</i>	10
Figura 6 – Verificação dos MAC <i>times</i>	12
Figura 7 – Inspeção do arquivo1.txt	12
Figura 8 – Atualização do <i>atime</i>	13
Figura 9 – Edição de arquivo1.txt.....	13
Figura 10 – Atualização <i>mtime</i>	14
Figura 11 – Atualização dos metadados	14
Figura 12 – Atualização do <i>ctime</i>	15
Figura 13 – Visualização pela interface gráfica.....	15
Figura 14 – Atualização do <i>atime</i> (2).....	16
Figura 15 – <i>Snapshot</i> 1	20
Figura 16 – Copiando o MBR.....	21
Figura 17 – Estrutura do MBR.....	22

LISTA DE QUADROS

Quadro 1 – Estrutura do MBR	20
-----------------------------------	----

EMENDAS

LISTAGEM DE COMANDOS DE PROMPT DO SISTEMA

Comando de prompt 1 – Criação de arquivo1.txt..... 12

EMSE

SUMÁRIO

PERÍCIA FORENSE POST-MORTEM.....	6
1 PERÍCIA <i>POST-MORTEM</i>	6
2 O QUE SE PODE ENCONTRAR EM ANÁLISES <i>POST-MORTEM</i>	7
3 RECUPERANDO ARQUIVOS.....	8
3.1 As assinaturas dos arquivos	8
4 A CRONOLOGIA DOS ACONTECIMENTOS – MAC TIMES	11
4.1 O <i>access time (atime)</i>	11
4.2 O <i>modification time (mtime)</i>	13
4.3 O <i>change time (ctime)</i>	14
4.4 Limitações dos MAC <i>times</i>	15
5 FERRAMENTAS FORENSES EM <i>SOFTWARE LIVRE</i>	16
5.1 DEFT Linux	16
5.2 CAINE	17
5.3 SIFT.....	18
5.4 The Sleuth Kit, Autopsy e mac-robber.....	18
5.5 Imagens forenses físicas e lógicas.....	19
6 HANDS-ON: DUPLICAÇÃO E VERIFICAÇÃO DO MBR.....	20
REFERÊNCIAS.....	23

PERÍCIA FORENSE POST-MORTEM

1 PERÍCIA *POST-MORTEM*

Para continuar na busca do *insider* que está prejudicando o seu cliente, cabe a você desenvolver, também, a habilidade dentro de um ambiente *post-mortem*.

A perícia forense *post-mortem* remete à perícia das mídias e dispositivos de armazenamento, a partir dos quais, a informação eletronicamente armazenada (*electronically stored information* – ESI) poderá ser recuperada. Vale lembrar que, de acordo com a *Federal Rules of Civil Procedure* (FRCP), a informação eletronicamente armazenada é definida como a informação criada, manipulada, comunicada, armazenada e mais bem utilizada em formato digital, exigindo o uso de *hardware* e *software*.

Os exemplos mais comuns relacionados a tais mídias e dispositivos vão dos CDs e DVDs – já considerados ultrapassados e, por isso, com utilização já relativamente reduzida – chegando aos pendrives e discos rígidos externos (HDS externos), dispositivos mais práticos e fáceis de usar, geralmente, com capacidade de armazenamento também bastante superior em relação às citadas mídias óticas.

Em artigo publicado na *Internet*, em agosto de 2018, a revista Época afirma que peritos da Polícia Federal produziram por volta de 1.600 laudos no âmbito da Lava Jato desde o advento da operação, em março de 2014, sendo a maior parte desses laudos relacionada a dispositivos de armazenamento informático, notebooks e outros equipamentos de guarda de dados – categoria em que se enquadram cerca de 1.200 dos laudos produzidos.

Segundo Barbosa e Gusmão (2016), esses dispositivos se tornaram essenciais para elucidação dos mais diversos tipos de investigação como pornografia infanto-juvenil, assédio moral e evasão de divisas, dentre outras.

As mídias de armazenamento, também referidas como memória auxiliar ou memória secundária, têm como uma de suas principais características a baixa volatilidade, o que, de maneira simplificada, significa que os dados armazenados nessas mídias são preservados mesmo após sua remoção ou após o desligamento do computador.

2 O QUE SE PODE ENCONTRAR EM ANÁLISES *POST-MORTEM*

A análise *post-mortem* tem como objetos de trabalho:

- Os arquivos armazenados no objeto questionado ou fragmentos de arquivos.
- Arquivos de hibernação.
- Os arquivos e/ou partições da memória virtual (*swap*), dentre outros.

Vale destacar, em complemento a esses dois últimos, a análise do conteúdo da RAM (*in vivo*).

Não compete ao perito/analista forense a recuperação de arquivos para produção, por exemplo: recuperar uma planilha de cálculos acidentalmente deletada pelo usuário; mas sim, a recuperação do arquivo – ou de fragmentos desse, para extração de evidências digitais relacionadas à demanda em curso.

De acordo com Eleutério e Machado (2014), ao apagarmos um arquivo do computador, o sistema operacional não o remove imediatamente do dispositivo, mas desaloca o espaço por ele utilizado, disponibilizando-o para gravação de novos arquivos, quando necessário. Adicionalmente, o nome desse arquivo também não aparecerá mais ao listar o conteúdo do diretório.

Assim sendo, é plausível afirmar que quanto antes o (conteúdo do) objeto questionado for preservado, maior será a probabilidade de recuperação das evidências (arquivos) nele armazenadas, incluindo-se, aqui, a recuperação de arquivos excluídos, os quais, em muitas ocasiões, ainda conservam a totalidade da informação original.

A Figura “Representação dos arquivos em disco” ilustra a visão desses autores ao compararem os arquivos armazenados em disco rígido à estrutura interna do planeta Terra.

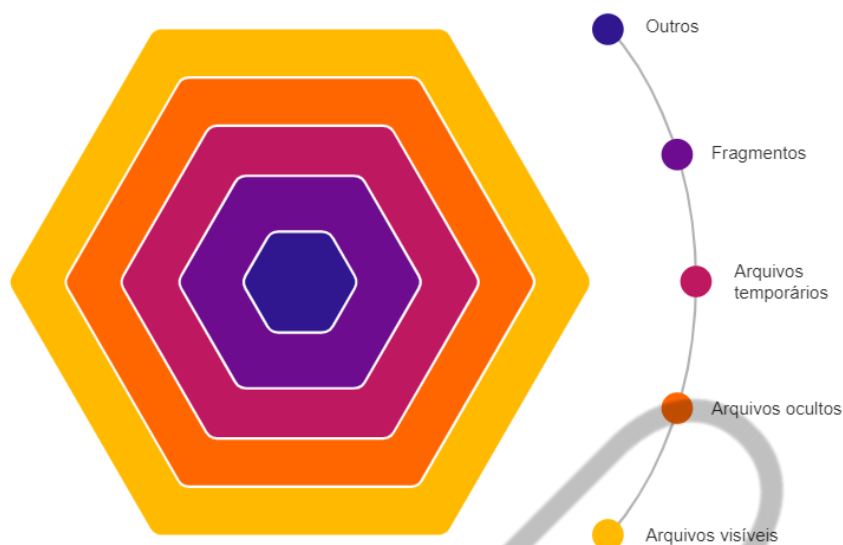


Figura 1 – Representação dos arquivos em disco
Fonte: Adaptado pelo autor com base em Eleutério e Machado (2014)

3 RECUPERANDO ARQUIVOS

Ainda de acordo com Eleutério e Machado (2014), a recuperação de arquivos tem como base as assinaturas de cada um desses. Tais assinaturas são também referidas como *magic numbers*, os quais consistem dos **quatro bytes iniciais do cabeçalho do arquivo**. Um dos grandes desafios da análise forense computacional recai sobre a identificação do tipo de arquivo, especialmente caso tenha sido deletado ou tenha sua extensão alterada.

3.1 As assinaturas dos arquivos

É relativamente comum que um usuário venha a alterar completamente o nome de algum documento (incluindo extensão) que deseja retirar furtivamente da empresa, por exemplo, via *e-mail* e sem deixar algum tipo de vestígio relacionado a tal ação. A identificação manual da assinatura (*magic number*) de um arquivo pode ser bastante simples. Considere-se a Figura “Imagem de referência”, inicialmente denominada *fiapon.jpg*.

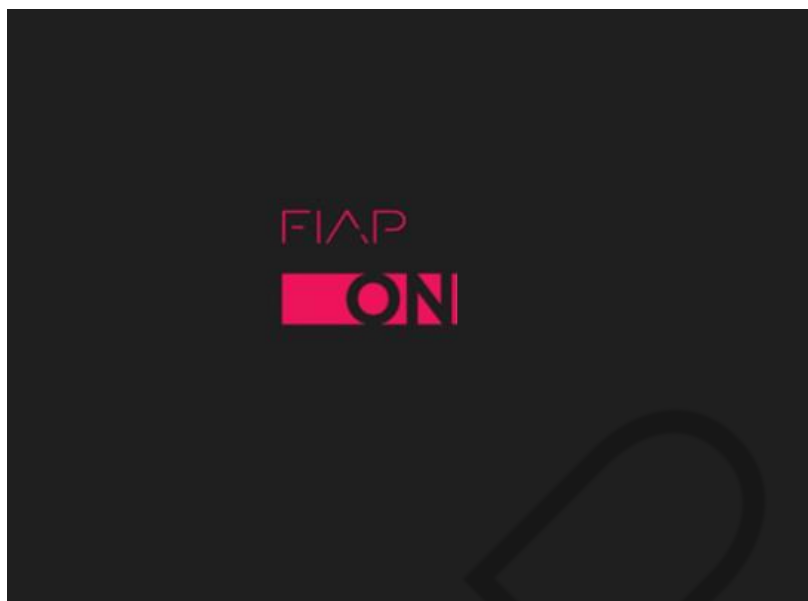


Figura 2 – Imagem de referência
Fonte: FIAP (2019)

Com a ajuda da ferramenta *file*, a partir de um terminal Linux é possível identificar facilmente o tipo de arquivo ao qual a imagem de referência pertence, mesmo que a extensão do arquivo venha a ser alterada ou removida, conforme ilustrado na Figura “Análise da imagem de referência”.

```
user1@investigator:~/Evidencias$ ls -l
total 16
-rw-r--r-- 1 user1 user 12319 Feb 19 00:19 fiapon.jpg
user1@investigator:~/Evidencias$ file fiapon.jpg
fiapon.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16,
baseline, precision 8, 512x512, frames 3
user1@investigator:~/Evidencias$
user1@investigator:~/Evidencias$ mv fiapon.jpg fiapon.pdf
user1@investigator:~/Evidencias$ file fiapon.pdf
fiapon.pdf: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16,
baseline, precision 8, 512x512, frames 3
user1@investigator:~/Evidencias$
user1@investigator:~/Evidencias$ mv fiapon.pdf fiapon
user1@investigator:~/Evidencias$ file fiapon
fiapon: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, base
line, precision 8, 512x512, frames 3
user1@investigator:~/Evidencias$
```

Figura 3 – Análise da imagem de referência
Fonte: Elaborado pelo autor (2019)

A partir da Figura “Análise da imagem de referência”, verifica-se que o nome original da imagem de referência: *fiapon.jpg* tem sua extensão confirmada pela saída da ferramenta *file*, ao ser executada pela primeira vez. Da mesma figura, pode-se observar que, mesmo após a imagem de referência ser renomeada para *fiapon.pdf*, a ferramenta *file* continua indicando que o tipo desse arquivo é uma imagem JPEG, o

que é mais uma vez confirmado após a terceira execução da ferramenta, dessa vez com o arquivo novamente renomeado, simplesmente, para fiapon. Tal fato decorre da assinatura do arquivo, conforme ilustrado na Figura “Assinatura do arquivo imagem de referência”.

```
user1@investigator:~/Evidencias$ hexdump fiapon | head -n3
00000000 d8ff e0ff 1000 464a 4649 0100 0001 0100
00000100 0100 0000 dbff 4300 0600 0504 0506 0604
00000200 0506 0706 0907 0a08 0a10 090a 0a09 0e14
user1@investigator:~/Evidencias$
```

Figura 4 – Assinatura do arquivo imagem de referência
Fonte: Elaborado pelo autor (2019)

A fim de obter-se a correta assinatura do arquivo (*magic number*), os quatro primeiros *bytes* do cabeçalho desse deverão ser **adequadamente reordenados, uma vez que sistemas baseados em microprocessadores INTEL armazenam informações em little endian.**

Do destaque na figura “Assinatura do arquivo Imagem de referência”, observa-se que a assinatura do arquivo será **ff d8 ff e0**, correspondendo ao formato JPEG, conforme ilustrado pela Figura “Verificação do *magic number*”.

6 Results Found For FFD8FFE0		
Extension	Signature	Description
★ JFIF	FF D8 FF E0	JPEG IMAGE
	ASCII	Size: 4 Bytes Offset: 0 Bytes
★ JPE	FF D8 FF E0	JPEG IMAGE
	ASCII	Size: 4 Bytes Offset: 0 Bytes
★ JPEG	FF D8 FF E0	JPEG IMAGE
	ASCII	Size: 4 Bytes Offset: 0 Bytes
★ JPG	FF D8 FF E0	JPEG IMAGE
	ASCII	Size: 4 Bytes Offset: 0 Bytes
★ JFIF	FF D8 FF E0	JFIF IMAGE FILE - jpeg
	ASCII	Size: 4 Bytes Offset: 0 Bytes
★ JPE	FF D8 FF E0	JPE IMAGE FILE - jpeg
	ASCII	Size: 4 Bytes Offset: 0 Bytes

Figura 5 – Verificação do *magic number*
Fonte: Filesignatures.net (2019)

4 A CRONOLOGIA DOS ACONTECIMENTOS – MAC TIMES

De acordo com Farmer e Venema (2011): “Às vezes saber quando algo aconteceu é mais valioso do que saber o que aconteceu.”, e MAC *times* são os atributos dos arquivos que auxiliarão na construção da cronologia. O *mtime* (*modification time*) registra a data em que o conteúdo do arquivo foi modificado pela última vez.

O *atime* (*access time*) registra a data em que o arquivo foi aberto para leitura pela última vez. Já o *ctime* tem interpretações diferentes no *Windows* e no *Linux/Unix*, o que pode levar a uma apresentação incorreta desse, por exemplo, quando um arquivo criado no *Windows* for acessado por meio de um sistema *Unix* ou vice-versa. No *Windows*, o *ctime* registra a data de criação do arquivo (*creation time*), enquanto no *Linux/Unix*, esse atributo registra a data de última alteração de algum metadado do arquivo (por exemplo, permissões de acesso desse). Sistemas *Unix* operam dessa forma, pois o tempo de criação (*creation time*) não é um requisito no POSIX. Sistemas *Windows* e alguns *Macintosh* baseados no *UNIX* apresentam ainda o *birth time* (*btime*), o qual faz referência à data de criação do arquivo.

É importante reforçar, com base no exposto que, de maneira geral, os MAC *times* registram apenas o último evento monitorado (modificação, acesso ou alteração do arquivo).

4.1 O *access time* (*atime*)

O *atime* é um atributo que merece especial atenção na medida em que, caso seu tempo de atualização seja muito curto, a *performance* do sistema poderá ser severamente impactada pela sua constante atualização. Caso seu tempo de atualização seja muito longo, a precisão das trilhas de auditoria poderá ser comprometida. Assim sendo, recomenda-se que o tempo de atualização do *atime* não seja alterado, a menos que realmente necessário para manter o equilíbrio entre a *performance* do sistema e os registros por ele gerados, altamente relevantes para fins forenses.

A fim de melhor entender os MAC *times*, será criado um arquivo chamado *arquivo1.txt*, conforme indicado na Listagem “Criação de *arquivo1.txt*”.

```
#lista o conteúdo do arquivo README, redirecionado-o para um arquivo cha-  
mado arquivo1.txt  
user1@investigator:~/Evidencias$ cat /usr/share/doc/bash/README >  
arquivo1.txt
```

Comando de prompt 1 – Criação de arquivo1.txt

Fonte: Elaborado pelo autor (2019)

A partir da Figura “Verificação dos MAC times”, é possível observar que os MAC times do arquivo são praticamente os mesmos, o que pode indicar ser esta a data e hora de criação do arquivo: dia 19/12/2019 10:05:35. Como se trata de um sistema *Linux*, conforme anteriormente exposto, o *btime* não foi registrado.

```
user1@investigator:~/Evidencias$ stat arquivo1.txt  
File: "arquivo1.txt"  
Size: 3839          Blocks: 8          IO Block: 4096   arquivo comum  
Device: 805h/2053d Inode: 1173         Links: 1  
Access: (0644/-rw-r--r--)  Uid: ( 1000/  user1)   Gid: ( 1000/  user1)  
Acces: 2019-02-19 10:05:35 .443057000 -0300  
Modify: 2019-02-19 10:05:35 .507057000 -0300  
Change: 2019-02-19 10:05:35 .507057000 -0300  
Birth: - ← btime não registrado  
user1@investigator:~/Evidencias █
```

Figura 6 – Verificação dos MAC times

Fonte: Elaborado pelo autor (2019)

Com ajuda do *vi*, um dos editores de texto mais tradicionais do *Linux*, vamos (apenas) inspecionar as primeiras linhas do conteúdo do arquivo, conforme a Figura “Inspeção do arquivo1.txt”.

```
Introduction  
*****  
  
This is GNU Bash, version 4.3. Bash is the GNU Project's Bourne  
Again Shell, a complete implementation of the POSIX shell spec,  
but also with interactive command line editing, job control on  
architectures that support it, csh-like features such as history  
substitution and brace expansion, and a slew of other features.  
For more information on the features of Bash that are new to this  
type of shell, see the file 'doc/bashref.texi'. There is also a  
large Unix-style man page. The man page is the definitive description  
of the shell's features.  
  
See the file POSIX for a discussion of how the Bash defaults differ  
from the POSIX spec and a description of the Bash 'posix mode'.  
  
There are some user-visible incompatibilities between this version  
of Bash and previous widely-distributed versions, bash-4.1 and  
bash-4.2. For details, see the file COMPAT. The NEWS file tersely  
lists features that are new in this release.  
  
Bash is free software, distributed under the terms of the [GNU] General  
Public License as published by the Free Software Foundation,  
version 3 of the License (or any later version). For more information,  
see the file COPYING.  
  
A number of frequently-asked questions are answered in the file  
'doc/FAQ'.  
  
:q█
```

Figura 7 – Inspeção do arquivo1.txt

Fonte: Elaborado pelo autor (2019)

Feito isto, basta digitar “:q” seguido da tecla <ENTER> para fechar o *vi*. Ao utilizar a ferramenta *stat* para verificação dos MAC *times* do arquivo, após esse acesso **apenas para leitura**, é possível constatar a atualização do *atime* (Figura “Atualização do *atime*”).

```
user1@investigator:~/Evidencias$ stat arquivo1.txt
  File: "arquivo1.txt"
  Size: 3839          Blocks: 8          IO Block: 4096  arquivo comum
Device: 805h/2053d   Inode: 1173         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  user1)   Gid: ( 1000/  user1)
Access: 2019-02-19 21:54:35.188056000 -0300
Modify: 2019-02-19 10:05:35.507057000 -0300
Change: 2019-02-19 10:05:35.507057000 -0300
 Birth: -
user1@investigator:~/Evidencias$
```

Figura 8 – Atualização do *atime*
Fonte: Elaborado pelo autor (2019)

Observe-se que a grande diferença verificada entre o valor do *atime* registrado quando da criação do arquivo, e o verificado após sua abertura para leitura, é propositalmente exagerada: o arquivo realmente foi criado às 10h05min e aberto apenas às 21h54min. Ainda da Figura “Atualização do *atime*”, pode-se verificar que o tamanho de arquivo1.txt é de 3839 bytes.

4.2 O *modification time (mtime)*

A fim de verificar o *mtime*, edite o arquivo1.txt, alterando a palavra *Introduction* logo no início do arquivo, para a palavra **Introducao**. Observe que os asteriscos que delimitam a palavra *Introducao* têm como função manter essa *string* com 12 caracteres (mesmo tamanho da *string Introduction*, preservando, dessa maneira, o tamanho original do arquivo). A Figura “Edição de arquivo1.txt” destaca a modificação descrita.

```
*Introducao*
=====

This is GNU Bash, version 4.3. Bash is the GNU Project's Bourne
Again SHell, a complete implementation of the POSIX shell spec,
but also with interactive command line editing, job control on
architectures that support it, csh-like features such as history
```

Figura 9 – Edição de arquivo1.txt
Fonte: Elaborado pelo autor (2019)

Após salvar o arquivo alterado, é possível verificar que também os MAC *times* do arquivo foram atualizados (Figura “Atualização *mtime*”).

```
user1@investigator:~/Evidencias$ stat arquivo1.txt
File: "arquivo1.txt"
  Size: 3839          Blocks: 8          IO Block: 4096   arquivo comum
Device: 805h/2053d   Inode: 1173         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  user1)   Gid: ( 1000/  user1)
Access: 2019-02-21 01:15:18.614019000 -0300
Modify: 2019-02-21 01:15:26.794019000 -0300
Change: 2019-02-21 01:15:26.794019000 -0300
 Birth: -
user1@investigator:~/Evidencias$ █
```

Figura 10 – Atualização *mtime*
Fonte: Elaborado pelo autor (2019)

A partir da Figura “Atualização *mtime*” é possível observar que arquivo1.txt foi aberto somente para leitura à 01h15min18, sendo fechado e novamente aberto, tendo seu conteúdo modificado à 01h15min26. Embora o tamanho do arquivo permaneça inalterado, a modificação em seu conteúdo atualizou também o metadado referente à data de criação ou de última modificação do arquivo, atualizando também o *ctime* (destaque na Figura “Atualização dos metadados”).

```

Tamanho do arquivo      Data da última
                           modificação
user1@investigator:~/Evidencias$ ls -l arquivo1.txt
-rw-r--r-- 1 user1 user1 3839 Feb 21 01:15 arquivo1.txt
user1@investigator:~/Evidencias$ █
```

Figura 11 – Atualização dos metadados
Fonte: Elaborado pelo autor (2019)

4.3 O *change time* (*ctime*)

Lembrando que o *ctime* é atualizado sempre que um metadado do arquivo é alterado, pode-se fazer a prova do conceito, por exemplo, alterando-se as permissões do arquivo, conforme ilustrado na Figura “Atualização do *ctime*”.

```
user1@investigator:~/Evidencias$ chmod g+w arquivo1.txt
user1@investigator:~/Evidencias$ ls -l arquivo1.txt
-rw-rw-r-- 1 user1 user1 3839 Feb 21 01:15 arquivo1.txt
user1@investigator:~/Evidencias$ stat arquivo1.txt
  File: "arquivo1.txt"
  Size: 3839          Blocks: 8          IO Block: 4096   arquivo comum
Device: 805h/2053d   Inode: 1173         Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   user1)   Gid: ( 1000/   user1)
Access: 2019-02-21 01:15:18.614019000 -0300
Modify: 2019-02-21 01:15:26.794019000 -0300
Change: 2019-02-21 01:48:50.674019000 -0300
 Birth: -
user1@investigator:~/Evidencias$
```

Figura 12 – Atualização do *ctime*
Fonte: Elaborado pelo autor (2019)

Da Figura “Atualização do *ctime*”, observa-se que o usuário *user1* acrescentou permissão de escrita no arquivo aos integrantes de seu grupo primário (*chmod g+w*), atualizando assim o *ctime*. Observe-se que tanto o *atime* quanto o *mtime* permaneceram inalterados.

4.4 Limitações dos MAC times

Os MAC times são muito úteis, especialmente para a construção de *timelines* (cronologias). Todavia, são também muito voláteis e facilmente fraudáveis. Por exemplo, observe-se que o simples fato de visualizar o arquivo por meio da *interface* gráfica do sistema pode alterar o *atime* (Figura “Visualização pela *interface* gráfica”).



Figura 13 – Visualização pela interface gráfica
Fonte: Elaborado pelo autor (2019)

Observe que tal fato não necessariamente ocorrerá com toda e qualquer *interface* gráfica.

A Figura “Atualização do *atime* (2)” ilustra a saída da ferramenta *stat*, empregada logo após a visualização do arquivo por meio da *interface* gráfica (o arquivo NÃO foi aberto). Conforme pode ser observado, o registro do *atime* foi alterado em relação ao anteriormente verificado na Figura “Atualização do *ctime*”, até então, o último acesso feito ao arquivo.

```
user1@investigator:~/Evidencias$ stat arquivo1.txt
  File: "arquivo1.txt"
  Size: 3839          Blocks: 8          IO Block: 4096   arquivo comum
Device: 805h/2053d   Inode: 1173         Links: 1
Access: (0664/-rw-rw-r-- )  Uid: ( 1000/  user1)   Gid: ( 1000/  user1)
Access: 2019-02-21 02:04:01.678019000 -0300
Modify: 2019-02-21 01:15:26.794019000 -0300
Change: 2019-02-21 01:48:50.674019000 -0300
 Birth: -
user1@investigator:~/Evidencias$ █
```

Figura 14 – Atualização do *atime* (2)
Fonte: Elaborado pelo autor (2019)

5 FERRAMENTAS FORENSES EM SOFTWARE LIVRE

Há várias distribuições Linux especializadas dedicadas à perícia forense computacional. A seguir, estão algumas referências.

5.1 DEFT Linux

Nascido a partir de uma ideia de Stefano Fratepietro, DEFT (sigla em inglês para *Digital Evidence & Forensics Toolkit*) é uma distribuição fabricada 100% na Itália, desenvolvida para forense digital e resposta a incidentes, com o propósito de ser executada no sistema analisado, sem adulterar ou corromper dispositivos (discos rígidos, *pen drives* e outros) conectados ao *host* no qual o processo de inicialização ocorre.

O DEFT é baseado no GNU Linux e pode ser executado em *live* (DVD ou *pendrive*) ou executado como um dispositivo virtual no VMware. Além de tudo isso, a equipe do DEFT dedica-se à implementação e ao desenvolvimento de aplicativos que são liberados para a empresa de consultoria *Digital Forensics and Mobile Forensics, Law Enforcement Officer* e os investigadores.

O DEFT é empregado em diversos lugares e por diferentes pessoas, como: militares, funcionários do governo, investigadores, auditores de TI e universidades, dentre outras.

5.2 CAINE

O CAINE (*Computer Aided Investigative Environment*) é outra distribuição italiana, *live*, do GNU / Linux criada como um projeto forense digital. À época da consulta ao *site*, o gerente do projeto era Nanni Bassetti (Bari, Itália). O CAINE oferece um ambiente forense completo, organizado para integrar ferramentas de *software* existentes como módulos e fornecer uma *interface* gráfica amigável. Os principais objetivos do CAINE recaem sobre:

- Um ambiente interoperável que suporte o investigador digital durante as fases da investigação digital.
- Uma interface gráfica amigável.
- Ferramentas de fácil utilização.

De acordo com o *site* do projeto, o “CAINE representa totalmente o espírito da filosofia *Open Source*, porque o projeto é totalmente aberto, todos podem assumir o legado do desenvolvedor anterior ou gerente de projeto” (NUNES, 2013). Ainda de acordo com o *site* do projeto, o lado Windows é *freeware* e, por último, mas não menos importante, a distro é instalável. Um dos destaques da versão 10 do CAINE, é que ele bloqueia todos os dispositivos de bloco (por exemplo, */dev/sda*), no modo *read only* por meio de uma ferramenta com GUI chamada *BlockON / OFF*.

Esse novo método de bloqueio de gravação garante que todos os discos sejam realmente protegidos contra operações de gravação acidentais, uma vez que se encontram bloqueados no modo somente leitura (*read only*). Se necessário escrever no disco, este poderá ser desbloqueado por intermédio do *BlockOn / Off* ou do *Mounter*, os quais poderão alterar a política de acesso ao disco para modo de escrita.

5.3 SIFT

O SIFT Workstation é um grupo de ferramentas gratuitas para resposta a incidentes e práticas forenses, projetadas para realizar exames forenses digitais detalhados em uma variedade de configurações. O SIFT oferece a capacidade de examinar com segurança os discos *raw* (entenda-se como a imagem forense do objeto questionado), múltiplos sistemas de arquivos e formatos de evidência. Ele coloca diretrizes rígidas em relação a como as evidências são examinadas (somente leitura), verificando se essas evidências não foram alteradas.

5.4 The Sleuth Kit, Autopsy e mac-robber

O *Sleuth Kit* (TSK) é uma biblioteca e coleção de ferramentas de linha de comando que permitem investigar imagens de disco. A principal funcionalidade do TSK permite analisar dados do volume e do sistema de arquivos. A estrutura de *plug-in* permite a incorporação de módulos adicionais para analisar o conteúdo de arquivos e construir sistemas automatizados.

A biblioteca pode ser incorporada em ferramentas forenses digitais maiores e as ferramentas de linha de comando podem ser usadas diretamente para encontrar evidências. Suas ferramentas permitem examinar sistemas de arquivos de um computador suspeito de maneira não intrusiva. Como as ferramentas não dependem do sistema operacional para processar os sistemas de arquivos, o conteúdo excluído e oculto é mostrado. Pode ser executado em plataformas *Windows* e *Unix*.

O *Sleuth Kit Framework* fornece uma plataforma aberta para os módulos da camada de aplicativos operarem. Os módulos não precisam se preocupar em obter acesso aos arquivos (o framework cuida disso) e os usuários não precisam se preocupar com dados de cópia entre várias ferramentas (o *framework* também cuida disso).

O TSK é frequentemente utilizado em conjunto com o *Autopsy*, uma plataforma forense digital e interface gráfica para o TSK e outras ferramentas forenses digitais. Ele é usado por policiais, militares e examinadores corporativos para investigar

incidentes de segurança ocorridos em computadores, podendo ser usado até mesmo para recuperação de fotos armazenadas em cartões de memória.

Outro projeto bastante relevante do *Sleuth Kit* é o *mac-robber*, ferramenta escrita em C utilizada em investigações digitais para coleta de dados de arquivos alocados em sistemas de arquivos montados. Isso é útil durante a resposta a incidentes, ao analisar um sistema ativo ou ao analisar um sistema inativo em laboratório. Os dados podem ser usados pela ferramenta *mactime* do TSK para criar uma linha do tempo da atividade do arquivo. O *mac-robber* exige que o sistema de arquivos seja montado pelo sistema operacional, ao contrário das ferramentas do TSK que processam o próprio sistema de arquivos. Portanto, o *mac-robber* não coletará dados de arquivos excluídos ou ocultos por *rootkits*, valendo enfatizar, também, que essa ferramenta modifica os *access times* em diretórios montados com permissões de gravação. O *mac-robber* é útil ao lidar com sistemas de arquivos não suportados pelo TSK ou por outras ferramentas de análise dos sistemas de arquivos.

5.5 Imagens forenses físicas e lógicas

Um dos principais objetos de trabalho da análise *post-mortem* recai sobre as imagens forenses, aqui simplificadaamente colocadas como uma cópia *bit-stream* do objeto questionado, constituindo uma reprodução absolutamente fiel deste, em forma de arquivo. Destaca-se, portanto, em relação isso, a existência de dois tipos de imagem forense:

- **Física:** executada pelo analista forense ao efetuar uma reprodução completa do objeto questionado, incluindo-se as suas áreas não abrangidas pelos sistemas de arquivos.
- **Lógica:** executada pelo analista forense ao efetuar uma reprodução parcial do objeto questionado, por exemplo, limitando-se a uma partição específica.

Recomenda-se que, sempre que possível, seja produzida a imagem física, uma vez que as imagens lógicas são frações dessas e poderão atrapalhar ou mesmo inviabilizar a investigação, caso venham a não atender às expectativas do investigador.

6 HANDS-ON: DUPLICAÇÃO E VERIFICAÇÃO DO MBR

O MBR (*master boot record*) é um tipo especial de setor de inicialização, localiza-se no início dos dispositivos de armazenamento em massa utilizados em sistemas compatíveis com IBM PC e demais. O MBR armazena as informações sobre como as partições lógicas, contendo sistemas de arquivos, são organizadas nessa mídia, armazenando, também, o código executável a ser utilizado para carregar o sistema operacional (*boot loader*).

A Figura “Estrutura do MBR” ilustra a estrutura do *master boot record*.

Endereço		Descrição	Tamanho em Bytes
Hex	Dec		
0000	0	Código de arranque do SO	440 (max. 446)
01B8	440	Assinatura opcional	4
01BC	444	normalmente nulo : 0x0000	2
01BE	446	Tabela de partições primárias (Quatro entradas de 16 bytes (IBM Partition Table scheme))	64
01FE	510	55h	2
01FF	511	AAh	
Tamanho total do MBR : 440 + 4 + 2 + 64 + 2 =			512

Quadro 1 – Estrutura do MBR
Fonte: Elaborado pelo autor (2019)

Antes de interagir com o MBR, é recomendável que seja criado um *snapshot* do estado atual da máquina virtual a ser utilizada para os testes (Figura “Snapshot 1”).

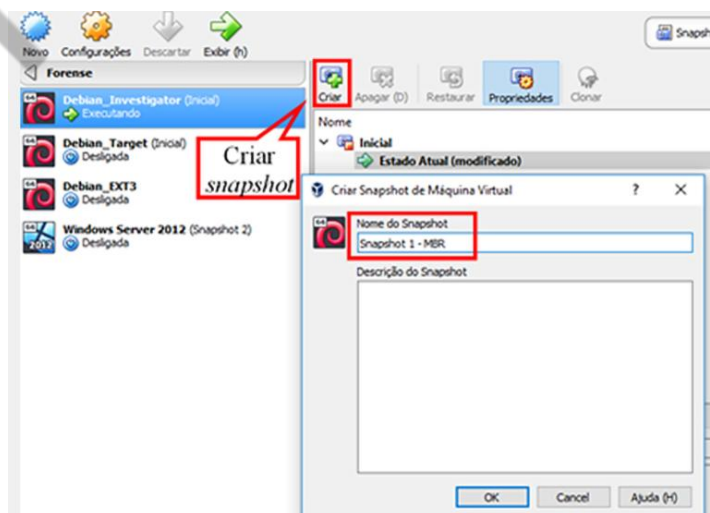


Figura 15 – Snapshot 1

Fonte: Elaborado pelo autor (2019)

Caso algo dê errado em relação ao procedimento de *backup* e verificação do MBR, prejudicando, eventualmente, o funcionamento da máquina virtual, seu estado imediatamente anterior ao problema poderá ser facilmente restaurado, permitindo que nova tentativa possa ser realizada.

A Figura “Copiando o MBR” ilustra o procedimento adotado.

```
root@investigator:/home/user1/Evidencias# dd if=/dev/sda of=mbr.raw bs=512 count=1
1+0 registros de entrada
1+0 registros de saída
512 bytes (512 B) copiados, 0,000397916 s, 1,3 MB/s
root@investigator:/home/user1/Evidencias# file mbr.raw
mbr.raw: DOS/MBR boot sector
root@investigator:/home/user1/Evidencias#
```

Figura 16 – Copiando o MBR
Fonte: Elaborado pelo autor (2019)

Na primeira linha de comando em destaque na Figura “Copiando o MBR”, a ferramenta ‘**dd**’ foi utilizada para produção de uma cópia *bit stream* do primeiro setor (**‘bs=512 count=1’**) do disco rígido (**‘/dev/sda’**), o qual contém o MBR. Ainda na mesma figura, observa-se que a ferramenta ‘**file**’ confirma ser o arquivo produzido (**mbr.raw**), o setor de *boot* do sistema (MBR).

A Figura “Estrutura do MBR” ilustra o conteúdo do arquivo *mbr.raw* (em formato hexadecimal + ASC II).

```

root@investigator:/home/user1/Evidencias# hexdump -C mbr.raw
00000000 eb 63 90 10 8e d0 bc 00 b0 b8 00 00 8e d8 8e c0 | .c..... |
00000010 fb be 00 7c bf 00 08 b9 00 02 f3 a4 ea 21 06 00 | .....!.. |
00000020 00 be be 07 38 04 75 0b 83 c8 10 81 fe fe 07 75 | ....8.u...u |
00000030 f3 eb 16 b4 02 b0 01 bb 00 7c b2 80 8a 74 01 8b | .....t... |
00000040 4c 02 cd 13 ea 00 7c 00 00 eb fe 00 00 00 00 00 | L..... |
00000050 00 00 00 00 00 00 00 00 00 00 80 01 00 00 00 | ..... |
00000060 00 00 00 00 ff fa 90 90 f8 c2 80 74 05 f8 c2 70 | .....t...p |
00000070 74 02 b2 80 ea 79 7c 00 00 31 c0 8e d8 8e d0 bc | t...y|..1... |
00000080 00 20 fb a0 64 7c 3c ff 74 02 88 c2 52 bb 17 04 | .....d|<t...R... |
00000090 f8 07 03 74 06 be 88 7d e8 17 01 be 05 7c b4 41 | .....t...}....A |
000000a0 bb aa 55 cd 13 5a 52 72 3d 81 fb 55 aa 75 37 83 | ..U..ZR=..U..u7. |
000000b0 e1 01 74 32 31 c0 89 44 04 40 88 44 ff 89 44 02 | .....21..D..@..D.. |
000000c0 c7 04 10 00 66 8b 1e 5c 7c 66 89 5c 08 66 8b 1e | .....f..\\f..\\f.. |
000000d0 80 7c 66 89 5c 0c c7 44 06 00 70 b4 42 cd 13 72 | |f..\\D...p..B...r |
000000e0 05 bb 00 70 eb 76 b4 08 cd 13 73 0d 5a 84 d2 0f | ...p..v.....s...z... |
000000f0 83 d0 00 be 93 7d e9 82 00 66 0f b6 c8 88 64 ff | .....}....f....d... |
00000100 40 66 89 44 04 0f b6 d1 c1 e2 02 88 e8 88 f4 40 | @f..D.....f...f.. |
00000110 89 44 08 0f b6 c2 c0 e8 02 66 89 04 66 a1 60 7c | ..D.....f..f..f.. |
00000120 66 09 c0 75 4e 66 a1 5c 7c 66 31 d2 66 f7 34 88 | f..uNf..\\f1..f..4... |
00000130 d1 31 d2 66 f7 74 04 3b 44 08 7d 37 fe c1 88 c5 | ..1..f..t..D..}7.... |
00000140 30 c0 c1 e8 03 08 c1 88 d0 5a 88 c8 bb 00 70 8e | 0.....Z.....p... |
00000150 c3 31 db b8 01 02 cd 13 72 1e 8c c3 80 1e b9 00 | ..1.....r..... |
00000160 01 8e db 31 f8 bf 00 80 8e c8 fc f3 a5 1f 61 ff | ...1.....a... |
00000170 26 5a 7c be 8e 7d eb 03 be 9d 7d e8 34 00 be a2 | &Z|.....}....4... |
00000180 7d e8 2e 00 cd 18 eb fe 47 52 55 42 20 00 47 65 | }.....GRUB...Ge |
00000190 6f 6d 00 48 61 72 64 20 44 69 73 6b 00 52 65 61 | om..Hard Disk..Rea |
000001a0 64 00 20 46 72 72 6f 72 0d 0a 00 bb 01 00 b4 0e | ..d..Error..... |
000001b0 cd 10 ac 3c 00 75 f4 c3 39 f6 5f d6 00 00 80 20 | !..<..u...9..... |
000001c0 21 00 83 56 38 1e 00 08 00 00 00 68 07 00 00 56 | !..V8.....h...V |
000001d0 39 1e 83 ae aa 04 00 70 07 00 00 38 77 00 00 ae | 9.....p...8w... |
000001e0 ab 04 83 ca 93 f7 00 a8 7e 00 00 98 3b 00 00 ea | .....~..... |
000001f0 b2 f7 05 fe ff fe 47 ba 00 02 b0 45 00 55 aa | .....6....E..U.. |
00000200
root@investigator:/home/user1/Evidencias#

```

Figura 17 – Estrutura do MBR
Fonte: Elaborado pelo autor (2019)

Com auxílio da ferramenta *hexdump*, com base na Figura “Estrutura do MBR”, verifica-se que a assinatura do MBR é **0xAA55** (a ordem deve ser invertida, pois processadores Intel utilizam armazenamento *little endian*). Ainda na referida figura, pode ser observado ser o GRUB o *boot loader* utilizado pelo sistema, conforme descrito a partir do *offset* 0x188. É importante destacar que o GRUB não utiliza o espaço entre os *offsets* 1B8h e 1BBh, por ser reservado à *Microsoft* para armazenamento do *NT Drive Serial Number* no *Windows NT/2000/XP/2003*. Na Figura “Estrutura do MBR” o valor armazenado (6f 39 d6 5f) não representa um valor válido, uma vez que o disco nunca foi ocupado por um sistema *Windows*.

Assim como na perícia *in vivo*, também teremos a etapa dois para a perícia *post-mortem*, estamos quase lá!

REFERÊNCIAS

BARBOSA, R. S. B. G; GUSMÃO, L. E. M. **Tratado de computação forense**. Campinas: Millennium, 2016.

CAINE. [s.d.]. Disponível em: <<https://www.caine-live.net/>>. Acesso em: 31 maio 2021.

EDRM. **EDRM Model**. [s.d.]. Disponível em: <<https://www.edrm.net/frameworks-and-standards/edrm-model/>>. Acesso em: 31 maio 2021.

ELEUTÉRIO, P. M. S.; MACHADO, M. P. **Desvendando a computação forense**. 4. Reimpressão. São Paulo: Novatec, 2014.

FARMER, D.; VENEMA, W. **Perícia Forense Computacional** – Teoria e Prática Aplicada. 4. Reimpressão. São Paulo: Pearson Prentice Hall, 2011.

NUNO, Nunes. Lançado CAINE 4.0, uma distribuição GNU/Linux focada em perícia forense e investigação. **Fórum Ubuntued**, 25 mar. 2013. Disponível em: <<http://forum.ubuntued.info/viewtopic.php?f=37&t=4403>>. Acesso em: 13 jun. 2022.

OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S. S. **Sistemas Operacionais**. 4. ed. Porto Alegre: Bookman e Instituto de Informática de UFRGS, 2010.

RAMOS, M. **Peritos da PF produzem 1,6 mil laudos no âmbito da Lava Jato**. [s.d.]. Disponível em: <[Peritos da PF produzem 1,6 mil laudos no âmbito da Lava Jato - Época \(globo.com\)](https://peritos.globo.com/peritos-da-pf-produzem-16-mil-laudos-no-ambito-da-lava-jato)>. Acesso em: 31 maio 2021.

SANS. **Sift Workstation**. Disponível em: <<https://digital-forensics.sans.org/community/downloads>>. Acesso em: 31 maio 2021.

SLEUTH KIT. [s.d.]. Disponível em: <<https://www.sleuthkit.org/>>. Acesso em: 18 fev. 2019.

VANDERBURG, E. **MAC Times in computer forensics**. 2009. Disponível em: <<https://www.tcdi.com/mac-times-in-computer-forensics/>>. Acesso em: 31 maio 2021.